

Programação Orientada a Objetos

Atributos, métodos e classes estáticas

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 **Introdução**
- 2 **Atributos Estáticos**
- 3 **Métodos Estáticos**
- 4 **Blocos Estáticos**
- 5 **Classes Aninhadas Estáticas**
- 6 **Exemplos e Padrões**



O que significa `static`?

- Em Java, o modificador `static` é usado para indicar que um membro (atributo, método ou classe aninhada) pertence à **classe** em si, e não a uma **instância** (objeto) específica.
- Isso significa que existe apenas uma cópia desse membro para todas as instâncias da classe.
- O membro estático é carregado na memória assim que a classe é carregada pela JVM.



- Também conhecidos como variáveis de classe.
- São declarados com a palavra-chave `static`.
- **Compartilhamento:** Todas as instâncias da classe compartilham a mesma variável. Se uma instância altera o valor, todas as outras veem a mudança.
- **Tempo de vida:** Existe durante toda a execução do programa (enquanto a classe estiver carregada).



Atributos de Instância

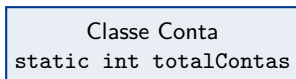
- Cada objeto tem sua própria cópia.
- Define o estado de um objeto individual.
- Criado quando o objeto é instanciado (`new`).

Atributos Estáticos

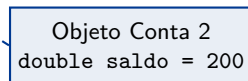
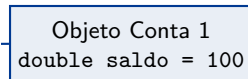
- Uma única cópia para a classe toda.
- Define propriedades comuns a todos os objetos.
- Criado quando a classe é carregada.



Method Area (Static Pool)



Heap Area



Ambos os objetos apontam para a mesma variável compartilhada na classe.

Exemplo de Código: Contador de Instâncias

```
1 public class Carro {  
2     public String modelo;  
3     public static int contadorProducao = 0; // Estatico  
4  
5     public Carro(String modelo) {  
6         this.modelo = modelo;  
7         contadorProducao++; // Incrementa para todos  
8     }  
9 }
```

- Toda vez que um `new Carro()` é chamado, o `contadorProducao` aumenta.
- Se criarmos 10 carros, o valor será 10 para todos.



- Embora seja possível acessar via instância, a boa prática é acessar via **Nome da Classe**.

```
1 Carro c1 = new Carro("Fusca");  
2  
3 // Forma recomendada (via classe)  
4 System.out.println(Carro.contadorProducao);  
5  
6 // Forma nao recomendada (via instancia)  
7 System.out.println(c1.contadorProducao);
```

Importante

O acesso via instância pode confundir outros programadores, dando a entender que a variável pertence apenas àquele objeto.



Constantes (`static final`)

- A combinação de `static` e `final` é usada para criar constantes em Java.
- `static`: Compartilhada (economiza memória).
- `final`: O valor não pode ser alterado após a inicialização.
- Nomenclatura sugerida: `CAIXA_ALTA` com underscores.

Exemplo

```
public static final double PI = 3.14159;
```



Quando usar Atributos Estáticos?

- ❶ **Contadores:** Quando você precisa saber quantos objetos foram criados.
- ❷ **Constantes:** Valores fixos que nunca mudam (PI, Taxas fixas).
- ❸ **Configurações Compartilhadas:** Como um tema de cores global ou um caminho de log.
- ❹ **Economia de Memória:** Evita duplicar dados idênticos em milhares de objetos.



O que são Métodos Estáticos?

- São métodos que podem ser chamados sem a necessidade de instanciar um objeto da classe.
- Pertencem à classe, não a um objeto individual.
- Marcados com a palavra-chave `static`.
- **Uso principal:** Operações utilitárias que não dependem do estado interno de um objeto.



- **Sem this:** Como não pertencem a uma instância, não existe a referência `this`.
- **Acesso limitado:** Só podem acessar diretamente outros membros estáticos da mesma classe.
- **Chamada:** Invocado usando `NomeDaClasse.nomeDoMetodo()`.

Sintaxe

```
public static int somar(int a, int b) { return a + b; }
```



Acesso a partir de...	Membro Estático	Membro de Instância
Método Estático	Permitido ✓	Proibido ✗
Método de Instância	Permitido ✓	Permitido ✓

Por que o Método Estático não acessa a Instância?

Porque o método estático pode ser executado sem que nenhum objeto tenha sido criado no "Heap". Ele não saberia para qual objeto olhar.



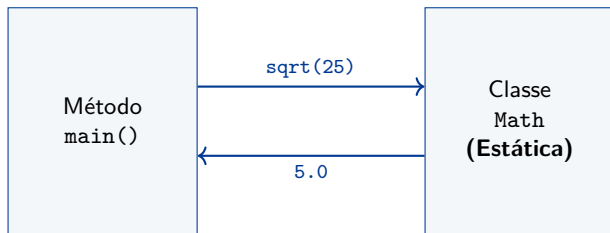
- Um dos maiores exemplos de uso de métodos estáticos no Java é a classe `Math`.
- Você não cria um "Objeto de Matemática", você apenas usa suas funções.

```
1 double raiz = Math.sqrt(25.0);  
2 double potencia = Math.pow(2, 10);  
3 int maior = Math.max(10, 20);
```

- Todos os métodos da classe `Math` são `static`.



Diagrama de Chamadas (TikZ)



*A chamada vai direto
para a definição da classe.*

- Usados para inicializar variáveis estáticas complexas.
- Executados apenas uma vez, quando a classe é carregada.

```
1 public class BancoDeDados {  
2     public static Connection conexao;  
3  
4     static {  
5         System.out.println("Carregando driver...");  
6         // Logica complexa de inicializacao aqui  
7         conexao = conectar();  
8     }  
9 }
```

- 1 Membros Estáticos e Blocos Estáticos (na ordem que aparecem).
- 2 Membros de Instância.
- 3 Construtor.

Importante

O bloco estático roda **antes** de qualquer objeto ser criado e apenas **uma vez** durante a vida da aplicação.



O que são Static Nested Classes?

- Uma classe definida dentro de outra classe, marcada como `static`.
- Diferente das *Inner Classes* (não-estáticas), a classe aninhada estática **não** possui uma referência para o objeto da classe externa.
- Comporta-se como uma classe de nível superior, mas está "escondida" dentro do namespace de outra classe.



Inner Class (Normal)	Static Nested Class
Precisa de uma instância da classe externa.	Não precisa de instância da classe externa.
Pode acessar todos os membros (até privados) da externa.	Só pode acessar membros estáticos da classe externa.
Declarada apenas como <code>class X</code> .	Declarada como <code>static class X</code> .

- Para criar uma instância de uma classe aninhada estática, usamos a sintaxe:

```
1 // Nao precisa de um objeto de 'Externa'
2 Externa.Aninhada objeto = new Externa.Aninhada();
```

- Comparação com Inner Class:

```
1 // Inner Class normal precisaria:
2 Externa ext = new Externa();
3 Externa.Interna in = ext.new Interna();
```



- **Agrupamento Lógico:** Se uma classe é usada apenas por outra, faz sentido mantê-las juntas.
- **Encapsulamento:** A classe aninhada pode ser privada, escondendo detalhes de implementação do mundo exterior.
- **Legibilidade:** Facilita encontrar códigos relacionados.



Exemplo: Estrutura de Dados Helper

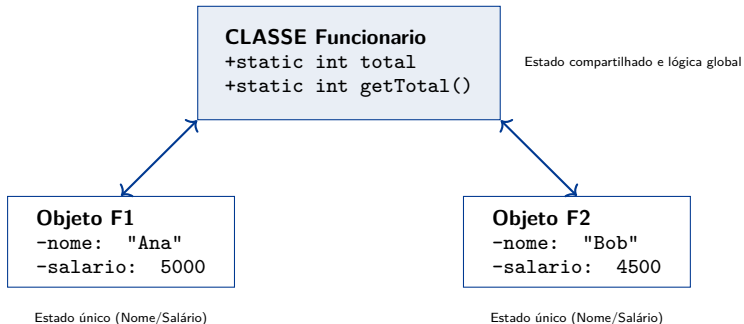
```
1 public class Lista {  
2     private static class No {  
3         int valor;  
4         No proximo;  
5         No(int v) { this.valor = v; }  
6     }  
7  
8     private No cabeca;  
9     // ... metodos da lista ...  
10 }
```

- O No não precisa saber qual Lista ele pertence individualmente, ele apenas armazena dados. Por isso ele é static.



Exemplo Completo: A Classe Funcionario

```
1 public class Funcionario {
2     private String nome;
3     private double salario;
4     private static int totalFuncionarios = 0; // Estatico
5
6     public Funcionario(String nome, double salario) {
7         this.nome = nome;
8         this.salario = salario;
9         totalFuncionarios++;
10    }
11
12    public static int getTotal() {
13        return totalFuncionarios;
14    }
15
16    public void exhibirDados() {
17        System.out.println(nome + " - RS " + salario);
18    }
19 }
```



- **Estado Global:** Evite usar atributos estáticos para armazenar dados que mudam frequentemente e afetam todo o sistema (dificulta o rastreamento de bugs).
- **Testabilidade:** Membros estáticos são difíceis de "mockar" em testes unitários.
- **Polimorfismo:** Se você precisa que o comportamento mude via herança, não use métodos estáticos.
- **Abuso de 'Utils':** Não transforme Java em uma linguagem procedural criando apenas classes cheias de métodos estáticos.



- ✓ `static` = Pertence à Classe.
- ✓ Atributos estáticos são compartilhados entre todos os objetos.
- ✓ Métodos estáticos não acessam `this` nem membros de instância.
- ✓ Blocos estáticos rodam uma única vez no carregamento.
- ✓ Classes aninhadas estáticas não precisam da classe externa instanciada.



Analise o código abaixo. Ele compila?

```
1 public class Teste {  
2     int x = 10;  
3     public static void main(String[] args) {  
4         System.out.println(x);  
5     }  
6 }
```

Resposta: Não!

O método `main` é **estático** e está tentando acessar a variável `x` que é de **instância** sem criar um objeto.

Perguntas?

Obrigado pela atenção!

